

UAV Mission Plan & Control Architecture

RobotX 2026 — Tasks 1–3 · ROS 2 **vision_geo** (perception + geolocation) and **control** (flight behaviour)

1. How the system works, in one paragraph

The UAV runs two ROS 2 nodes. **vision_geo** looks at the ground through a downward camera, runs a YOLO model (OpenCV DNN, ONNX) to recognise the objects of the current mission, and converts every detection from a *pixel* into a *GPS coordinate*. **control** flies the drone: take-off, survey pattern, hover, centering over a target, servo release, and return. The two nodes never block each other — vision keeps mapping while control keeps flying. All inter-vehicle traffic (UAV ↔ USV/UUV) is plain ROS 2 topics: object positions go out as NavSatFix-style GPS messages, and each mission ends with the UAV parking in a hover until the partner vehicle publishes the next "GO". One node binary, one YOLO model per mission — only the config file and the class list change.

2. Tech stack — what each piece actually does

Layer	Technology	What it does for us
Flight stack	ArduPilot + MAVROS (GUIDED mode)	Executes position/velocity setpoints and gives us GPS, local ENU pose, attitude quaternion and relative altitude.
Perception	YOLO ONNX via OpenCV cv::dnn	One model per mission. Outputs class id + confidence + bounding box at 320×320 or 640×640 input, NMS applied.
Geolocation	Pinhole ray-cast to ground plane	Pixel → camera ray (inverse K) → body frame → world frame (attitude quaternion) → intersect flat water at known AGL → East/North offset → lat/lon.
Mapping	Pixel tracker + running-mean map	Same object seen over many frames is one map point. Its GPS is the average of every observation, so the position keeps getting more accurate each pass.
Messaging	vision_msgs/ DetectedObjectArray	class_id, label, confidence, pixel u/v, latitude, longitude, north/east offset — one message carries the whole object list.
Flight logic	control_ library (C++)	takeoff, LocalMove, sendGlobalRaw, holdPosition, centering_object, controlServo, rotateByDegrees, setMode("rtl") — reusable blocks the mission script calls in order.
Monitoring	RViz2 MarkerArray + TF, OpenCV window	Judges and operators see the live map and the annotated camera feed.

3. Mission 1 — Safe Passage (UAV maps the buoy field for the USV)

Goal: the safe route is only visible from above. The UAV must find it and hand it to the USV. Model classes: buoy colours / beacon states (RED, GREEN, BLUE-flash = entry, BLUE-solid = exit, OFF/black).

Step	What the UAV does	Control function	Output / check
1. Launch	Arm, take off and climb to 15–20 m survey altitude, then hold until pose is stable.	takeoff() → holdPosition()	pose.z within tolerance
2. Survey	Fly a lawnmower pattern over the buoy field at constant altitude and level attitude.	LocalMove() / sendGlobalRaw()	waypoint reached
3. Detect	YOLO classifies each buoy and beacon state in every frame.	vision_geo::infer()	conf > threshold, NMS
4. Localise	Every detection is ray-cast to the water surface and turned into lat/lon.	projectPixelToGPS()	ray points down, AGL > 0.5 m
5. Map	Repeat sightings of the same buoy merge into one map point (2 m radius) and its GPS is re-averaged.	updateMarkers()	min. 5 observations before a buoy is committed
6. Report	Publish the finished buoy list (position + colour) to the USV and to RoboCommand.	detections_pub_	/vision_geo/detections
7. Hand over	Hover on station at the last survey point and wait for the next tasking.	holdPosition() loop	waits for mission "GO"

4. Mission 2 — Infrastructure Survey & Repair (UAV delivers the resource tin)

Goal: mark the GREEN start buoy for the UUV, then after the UUV repairs the pipeline, drop a tin on the colour circle it asks for. Model classes: colour indicator (red/green), platform, red/green/blue circle, tin.

Step	What the UAV does	Control function	Output / check
1. Resume	Leaves hover on "GO", climbs/transits to the pipeline survey area.	sendGlobalRaw()	target GPS reached
2. Mark start	Detects which of the two buoys shows the GREEN indicator and publishes its GPS to the UUV.	vision_geo pipeline	class = green_indicator
3. Wait	Hovers on station while the UUV surveys and repairs the pipeline.	holdPosition()	listens for the resource request
4. Receive task	Reads the requested tin colour + destination circle colour from the UUV message.	ROS 2 subscription	request topic received
5. Approach	Flies to the platform GPS produced by vision_geo, descends to drop altitude.	sendGlobalRaw() → LocalMove()	platform in frame
6. Align	Visual servoing: centres the drone directly over the correct colour circle using pixel error.	centering_object()	pixel error < tolerance, timeout guarded

Step	What the UAV does	Control function	Output / check
7. Drop	Fires the servo to release the tin, confirms release, reports completion.	controlServo()	servo ack + drop reported
8. Hand over	Climbs back to safe altitude and hovers, waiting for Mission 3 "GO".	holdPosition() loop	waits for mission "GO"

5. Mission 3 — Coordinated Logistics (retrieve the correct tin, deliver it)

Goal: after the USV docks and puts out the fire, it tells the UAV which tin colour is needed and which circle it goes to. The UAV must pick that exact tin up from one platform and deliver it to the other.

Step	What the UAV does	Control function	Output / check
1. Resume	Leaves hover on the USV tasking message; decodes tin colour + destination colour.	ROS 2 subscription	two-colour pattern decoded
2. Find source	Transits to the source platform and detects the tins; keeps only the requested colour class.	sendGlobalRaw() + vision_geo	correct tin classified
3. Pick up	Centres over the tin, descends, and closes the gripper / magnet.	centering_object() → Descend() → controlServo()	pickup confirmed, then climb
4. Deliver	Transits to the delivery platform, centres on the target colour circle, releases.	sendGlobalRaw() → centering_object() → controlServo()	release inside circle
5. Report	Publishes task-complete to the USV and RoboCommand.	publisher	completion message sent
6. Finish	Returns to launch and lands.	setMode("rtl")	disarmed on ground

6. Mission sequencing — how the three missions are chained

Every mission ends the same way: the UAV finishes its task, publishes its result, then enters a **hover-and-wait** state at either the last survey point or the staging point of the next mission. It only leaves that state when the partner vehicle publishes the mission "GO" topic. This makes the sequence *event-driven instead of time-driven*: if the USV or UAV is slow, the UAV simply keeps hovering; nothing desynchronises. The same pattern also lets a mission be re-run without restarting the whole system.

7. Robustness — what can go wrong and what we do about it

Risk	Handling in the code
Stale or missing GPS / pose	Every projection checks max_pose_age_ms (500 ms) and the pose-valid flags; stale frames are dropped instead of producing a wrong coordinate.
Drone too low or camera looking up	Projection is rejected if AGL < 0.5 m or the ground ray does not point downward — prevents divide-by-near-zero and coordinates at infinity.
False / flickering detections	A detection must survive min_obs_to_commit = 5 frames before it enters the map, and tracks get a track_exit_frames = 5 grace period for brief occlusions.
Same object counted twice	New map points merge into an existing one within map_merge_radius_m = 2 m of the same class.
Noisy single-frame position	The stored GPS is the running mean of every observation, so accuracy improves with each pass rather than jumping around.
Attitude tilt during flight	The attitude quaternion is applied to every ray, so roll/pitch/yaw while surveying does not bias the mapped position.
Centering never converges	All centering routines take a timeout; on expiry the drone falls back to a stable hover instead of drifting.
Inference / camera failure	The DNN forward has a fallback output path and throttled error logging; control keeps flying the planned pattern even with zero detections.
Loss of link / task failure	setMode("rtl") is the terminal fallback in the mission script — the drone always has a safe way home.

Implementation status. The vision_geo perception + GPS mapping pipeline and the control function library are built and running. The current src/control/src/mission/control.cpp is still a bring-up script (take-off → fixed local moves → RTL); the per-mission sequences and the "GO" handshake described in sections 3–6 are the target design and are the next thing to be wired into that file.